

Reporting Layer en PostgreSQL

Construyendo reportes limpios, reutilizables y verificados.



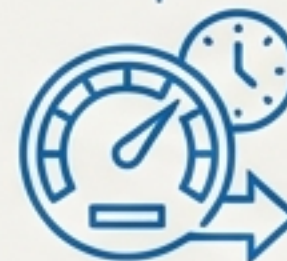
Meta: Que tu base de datos “publique” reportes listos para consumir.

Regla de Oro: Cada reporte declara su *grain* y trae evidencia (**VERIFY**).

Objetivos de la Sesión

De queries ad-hoc a ingeniería de datos.

Al finalizar esta sesión, transformarás consultas sueltas en una capa de reporte robusta y auditada.



1. **Definir el Grain:** Entender exactamente qué representa 1 fila.
2. **Lógica de Fila:** Crear campos calculados (CASE, COALESCE, CAST).
3. **Lógica de Grupo:** Escribir agregaciones robustas (GROUP BY / HAVING).
4. **Arquitectura:** Encapsular lógica en VIEWS ("Contratos").
5. **Performance:** Leer lo básico de EXPLAIN y aplicar índices útiles.

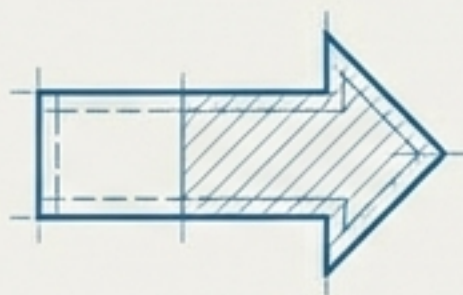
El Concepto de “Grain” (Granularidad)

Antes del SQL, pregúntate: ¿Qué representa 1 fila?



Raw Data (Orders)

1 Fila = 1 Producto Vendido



Date	Total Sales	Orders Count	Metric

Reporting Layer

1 Fila = Mes (YYYY-MM)

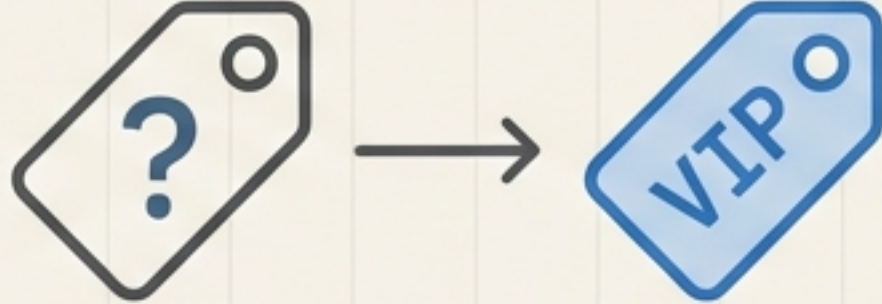
Warm-Up: Antes de escribir código

- **Grain:** Define la entidad única.
- **Métricas:** Define la operación (SUM, COUNT, AVG).
- **Verify:** Diseña 2–3 queries para comprobar el resultado.

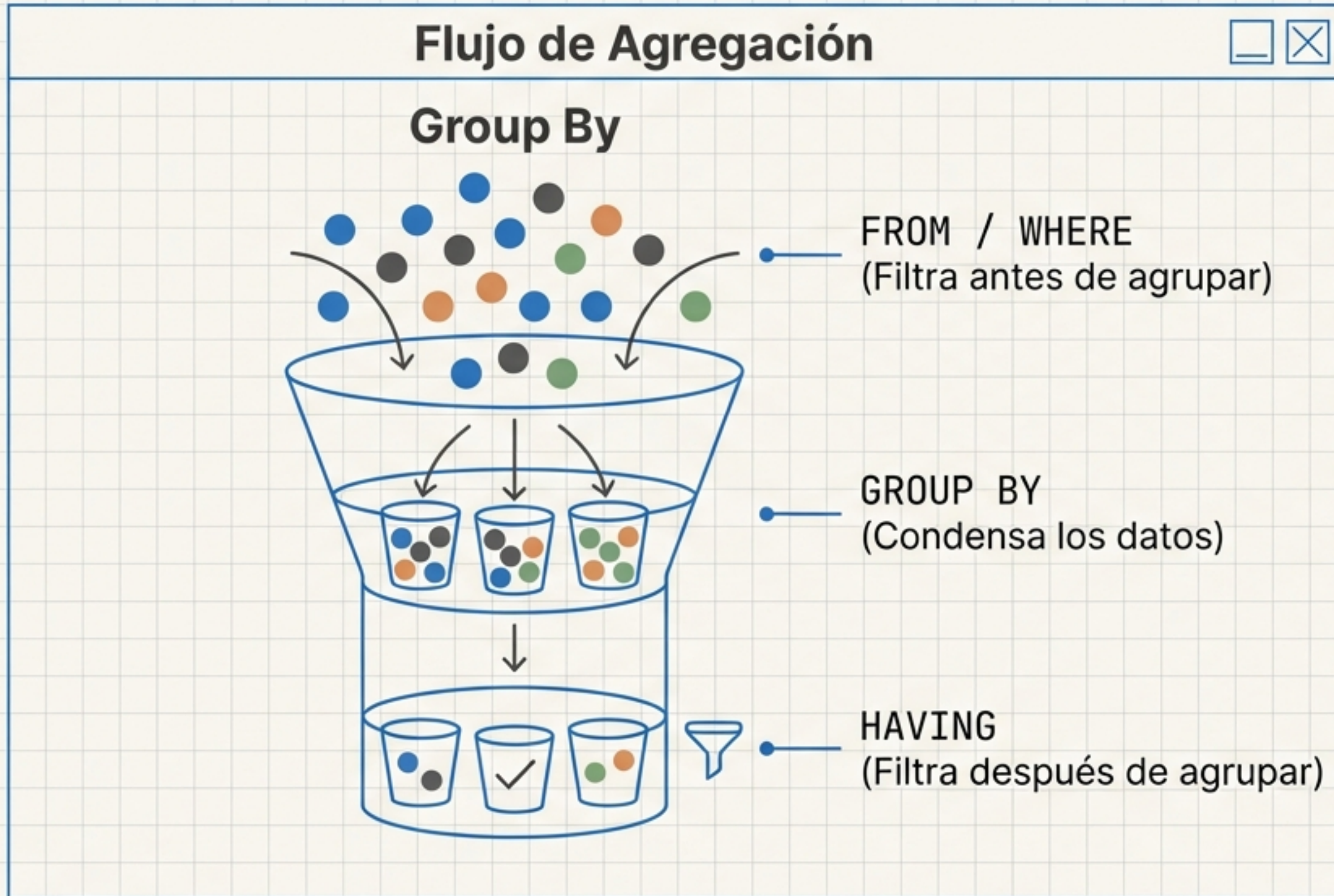
Ejemplos: Cliente, Orden, Mes, Producto.

Limpieza de Datos: Campos Calculados

Patrones de ingeniería para la vida real.

Classification <pre>CASE WHEN gasto > umbral THEN 'VIP' ELSE 'Regular' END</pre>	Clasificar banderas y estados.	
Null Safety <pre>COALESCE(subtotal, 0)</pre>	Convertir nulos a valores default para no romper matemáticas.	$\text{NULL} + \text{X} = \text{ERROR}$ $0 + 1 = 1 \checkmark$
Consistency <pre>CAST(valor AS numeric) / ::numeric ROUND(valor, 2)</pre>	Tipos de datos consistentes y redondeo.	

Agregación sin Errores



Rules & Best Practices

Regla de Oro: Si seleccionas una columna normal, debe estar en el GROUP BY.

`COUNT(\*)`: Cuenta filas (incluye nulos).

`COUNT(col)`: Cuenta valores (ignora nulos).

SUM (Ventas),
COUNT DISTINCT (Clientes),
AVG (Ticket).

Checklist Anti-Bugs

Errores comunes y cómo detectarlos.

- [] ¿Tu *grain* es consistente con el SELECT?
- [] ¿El COUNT está inflando números por repetición?
- [] ¿Los valores NULL están rompiendo tu AVG? (Tip: Usa COALESCE).
- [] ¿La fecha está agrupada correctamente? (No mezclar día con mes).

Alerta Roja

“Me dio más ventas que en la tabla original”

Causa: Duplicación de filas por un JOIN mal puesto (fan-out).

Blueprint 1: Top Productos por Ventas

SPECS

- **Grain:** 1 Fila = 1 Producto
- **Verification:** Validar un producto específico con suma manual.

CODE

JetBrains Mono

```
SELECT
    product_name,
    SUM(detalle.subtotal) AS total_vendido,
    COUNT(DISTINCT orden_id) AS num_ordenes
FROM orders
JOIN order_details ...
GROUP BY product_name
ORDER BY total_vendido DESC;
```

VISUAL TABLE

	Product Name	Total Sales	Order Count
1	Laptop Pro X	\$12,500	45
2	Monitor 4K	\$8,200	32
3	Mouse Wireless	\$1,200	150

Blueprint 2: Ventas Mensuales

SPECS

- **Grain:** 1 Fila = Mes (YYYY-MM)
- **Technique:** Construir el campo 'mes' desde created_at.

CODE

JetBrains Mono

```
SELECT
  TO_CHAR(created_at, 'YYYY-MM') AS mes,
  SUM(total_orden) AS total_ventas,
  COUNT(*) AS num_ordenes,
  AVG(total_orden) AS ticket_promedio
FROM orders
GROUP BY 1;
```

TIMELINE VISUALIZATION

Day 1
Day 3
Day 5
Day 4
Day 12
January

VISUAL TABLE

	Mes	Total Ventas	Num Ordenes	Ticket Promedio
1	2023-01	\$45,000	120	\$375
2	2023-02	\$41,200	110	\$374
3				

Views como Contratos de Datos



TEXT EXPLANATION

- **Definition:**

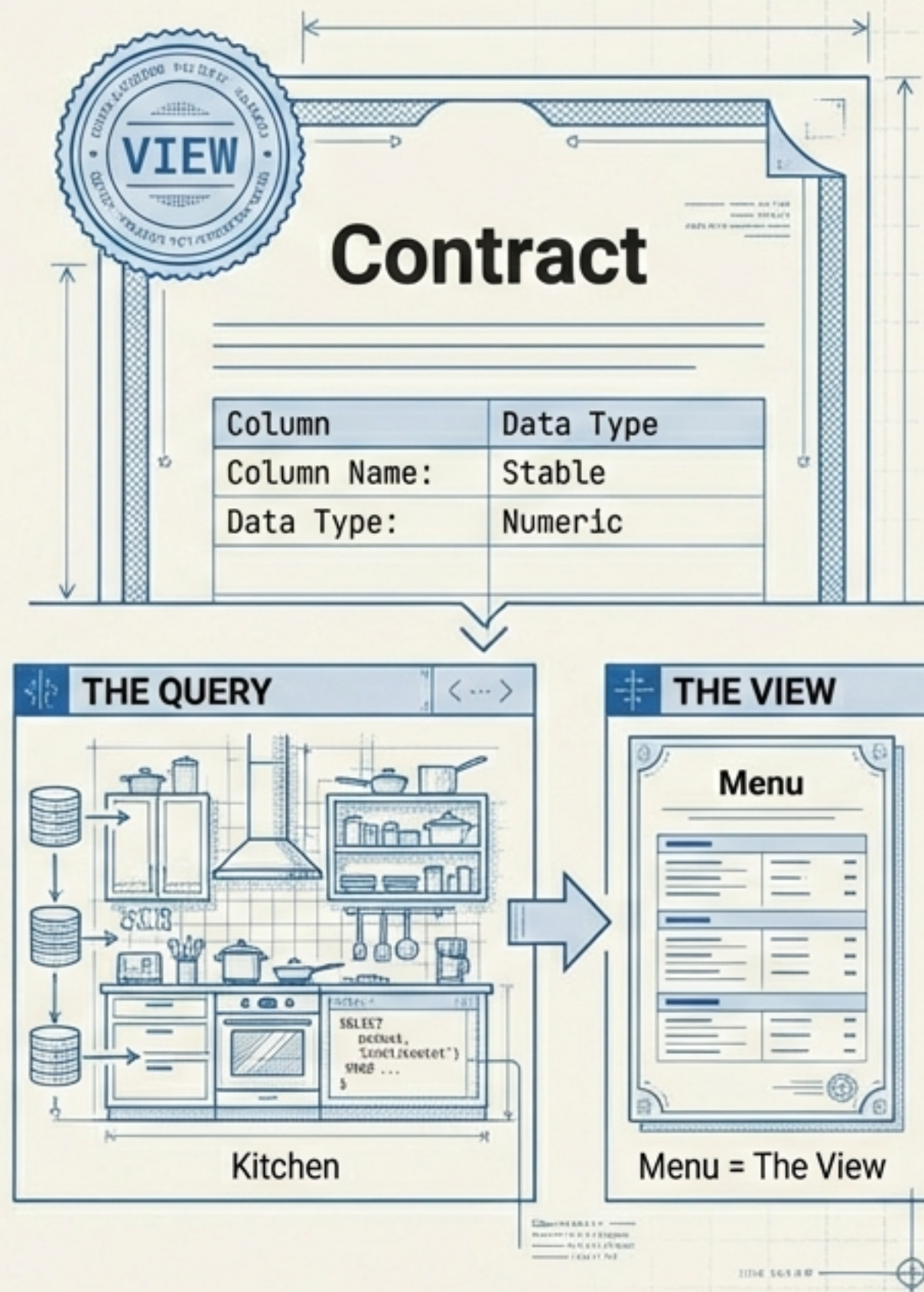
Una VIEW es una “consulta guardada” que actúa como capa de reporte reusable.

- **Why it Matters:**

- **Consistency:** API y Dashboards consumen la misma verdad.
- **Flexibility:** Cambia la query interna sin romper consumidores.

- **Best Practices:**

- **Naming:** ``vw_report_top_productos``
- **Explicit:** Evita ``SELECT *``. Declara columnas estables.



The VERIFY Mindset

Evidencia dentro del SQL. Evidence > Assumption.



Performance Check: EXPLAIN + Índices

Intro sin trauma.



Los índices no son magia. Son útiles para columnas en **WHERE**, **JOIN** y **ORDER BY**.



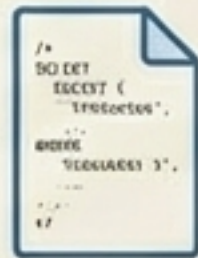
Mira siempre el **Rows Estimate** en el EXPLAIN.

Mini-Lab: Manos a la Obra

Objetivo: Publicar 2 reportes como VIEWS con evidencia y performance.

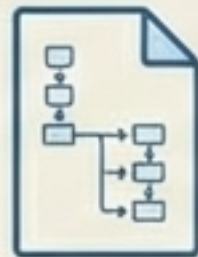


entregables/



views.sql

Top Productos + Ventas Mensuales



explain.sql

Antes vs Después del Index



README.md

Instrucciones + Grain Definition



La Defensa (2 min)

-  • Explica el **Grain**.
-  • Muestra 1 **Verify** que pruebe que no hay bugs.
-  • Justifica por qué elegiste ese **Índice**.